Software - Architecture, Design, Test, and...

Troels Mortensen

June 9, 2026

Contents

Todo list

Chapter 1

3-Layered Architecture

This chapter teaches *data-centric three-layered architecture*: a classic way to structure business applications where the relational database schema is the anchor and software is built in layers on top of it.[6, 7] We separate the system into *presentation*, *business logic*, and *data access*, so each part has a clear responsibility. In Java packages we often use the names `presentation`, `application` (for the business logic layer), and `persistence` (for the data access layer).

We apply this style in a **logical layered monolith**: one deployable application where packages or modules express layers. A formal definition of *layer* appears in a later part of this book. Physical deployment patterns are out of scope here.

IN-REF: write layer definition chapter (what-is-architecture)

In this chapter, we use the `Abyssal Watch` case study with two running examples: the **write** use case `Confirm First Kaiju Detection` (SCN-02) and the **read** use case `View Mission Kaiju Briefing`. If you read the 2-layered chapter first, you may have introduced a standalone `entities` module shared by presentation and persistence. In the classic three-layer style, table-shaped persistence entities live *inside* the data access layer instead; we explain that refinement in Section 1.1.6.

In the course that accompanies this book, you create database tables and write SQL yourself. The code snippets therefore use **persistence entities** (plain Java classes aligned to tables) and **JDBC DAOs** with hand-written `PreparedStatement` queries—not an ORM.

1.0.1 From Two Layers to Three

Chapter **??** kept business logic in presentation controllers (JavaFX or Web API) while persistence stored domain entities from a shared `entities` module. Three layers add an explicit `application` (business logic) layer: controllers stay thin, and services such as `ConfirmKaijuDetectionService` (Code snippet 1.7) own rules and transaction sequencing. Persistence entities move under `persistence` rather than a top-level `entities` module shared with presentation—see Section 1.1.6.

1.1 Why We Use Data-Centric Three Layers

The main benefit of this style is separation of concerns around **data management and entity flow**: how records are captured at the top, processed in the middle, and written to tables at the bottom.[7] We isolate user interaction in `presentation`, procedural rules and transaction sequencing in the business logic layer (`application`), and SQL and entity mapping in the data access layer (`persistence`).

The default dependency direction is:

- `presentation -> business logic -> data access -> database`

In practice:

- `presentation` translates user actions into calls the business layer understands and formats results for humans or APIs,
- the business logic layer sequences reads and writes, enforces rules, and opens transaction boundaries,
- the data access layer converts between table rows and code objects; the database remains the source of truth for stored state.

A useful mental model:[6]

- **Presentation** — the counter clerk: accepts requests and shows results, without deciding business outcomes,
- **Business logic** — the kitchen manager: coordinates work, applies rules, and decides what gets stored,
- **Data access** — the supply pantry: fetches and puts away stock according to storage layout, without interpreting policy.

1.1.1 Logical Layers vs Physical Tiers

Do not confuse the **logical layers** in this chapter with a **physical three-tier deployment**. Logical layers are package or module responsibilities inside one deployable application: presentation, business logic, and data access code living in the same process, with dependency rules between them. A physical three-tier setup places presentation, application, and database on separate machines or processes (for example a web server, an app server, and a database server). The names overlap, but the concerns differ: here we teach structure inside the monolith; how you deploy processes is a separate topic. Figure 1.1 summarizes the logical stack we build in this chapter.

DIAGRAM: three-layer stack inside a monolith boundary: presentation, application, persistence, database

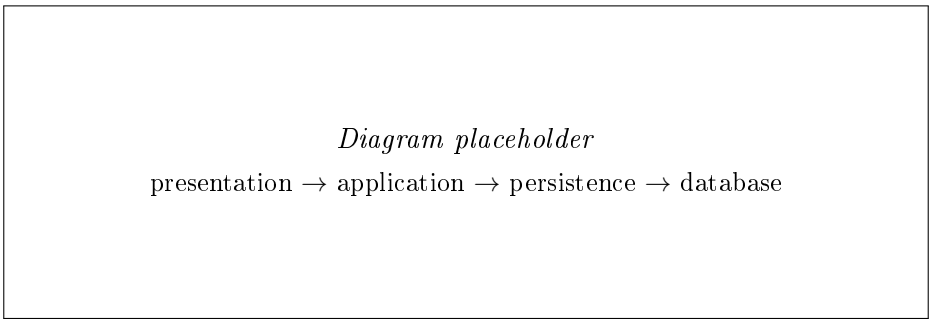

Figure 1.1: Logical three-layer stack inside one deployable application (diagram to be added).

1.1.2 When Data-Centric Three Layers Fit

This style is a strong default when:

- the system is CRUD-heavy and transactional, with a stable relational schema at the center,
- the team ships features bottom-up from tables and DAOs,
- onboarding benefits from predictable layer placement rules.

Signs you may be outgrowing the baseline (without abandoning layering):

- business service classes become oversized and hard to test in isolation,

- provider-shaped DAO interfaces fight the vocabulary your use cases need,
- relaxed read shortcuts spread until policy is unclear,
- public API contracts suffer on every schema rename.

1.1.3 Transitive Dependency and Layer Monopolies

Transitive dependency rule: presentation must not skip the business logic layer to call the data access layer directly, except in deliberate relaxed read paths (Section 1.1.4). Skipping layers hides dependencies and makes schema changes hard to contain.[6]

Monopoly rules:

- the business logic layer owns business decisions and transaction sequencing,
- the data access layer owns SQL, entity mapping, and persistence mechanics—not business rules,
- presentation owns interaction and formatting—not persistence or policy.

Fight leakage *downward* (rules buried in stored procedures) and *upward* (smart UI widgets or controllers that orchestrate storage).

1.1.4 Relaxed Read Paths

Default: read flows go through the business layer when policy, reuse across channels, or invariant checks matter.

Exception: read-only, presentational lookups with *no* business decision—for example static labels, reference data for dropdowns, or data whose rules were already enforced on a prior write. Allowed shapes include a thin read facade in `application` or, in the strictest relaxation, `presentation` calling a specialized read-only DAO or query object. There are still no writes and no transaction orchestration in presentation.

Risks include schema coupling, duplicated read logic, and policy drift when screens grow rules later. Mitigate with read projections owned by data access (or `dtos`) and revisit the path when business logic appears.

`Confirm First Kaiju Detection` is **not** a relaxed read. For `View Mission Kaiju Briefing`, the thinnest data-access-level assembly variant may use a relaxed path; see Section 1.8. Figure 1.2 contrasts the default write path with the relaxed read exception.

DIAGRAM: strict write path vs relaxed read exception (presentation to application to persistence vs presentation to read DAO)

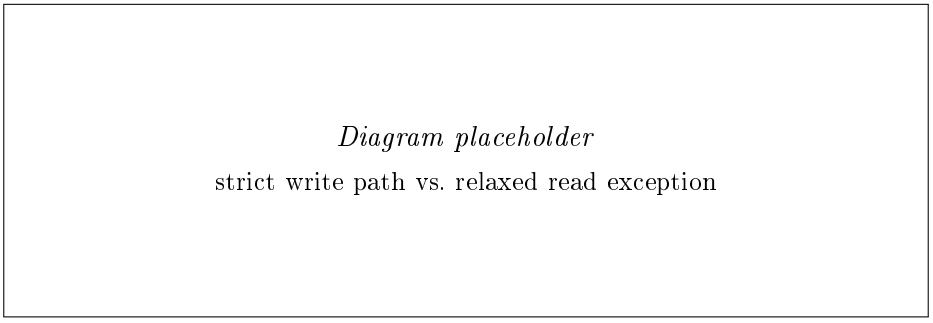

Figure 1.2: Default write path through the business layer vs. relaxed read paths (diagram to be added).

1.1.5 A Fowler Variation (Not This Chapter's Anchor)

Martin Fowler describes a layered enterprise model with presentation, *domain*, and data source.[2] The domain layer can hold rich behavior that is not tied one-to-one to tables. That variation is common and valuable, but it is not the data-centric workflow we teach here. This chapter stays **schema-first and bottom-up**; a later chapter on hexagonal architecture explores **behavior-first** design with explicit ports and adapters.

1.1.6 The Entities Refinement

We first describe the stack as:

- `presentation -> business logic -> data access -> database`

For clarity we often refine the bottom of the stack to:

- `presentation -> business logic -> data access -> entities -> database`

Entities are not a fourth deployable tier and presentation must not call them directly. They are **table-shaped persistence entities**—plain Java classes whose fields align with columns—that the data access layer maps to and from the database using SQL you write. The business layer manipulates those types *through* data access APIs such as `KaijuDao` (Code snippet 1.2), not by reaching past the data access layer into storage.

In packages, entities belong *inside* data access, for example `persistence.entities`, not as a top-level sibling module at the solution root (contrast Chapter **??**, where a separate `entities` module was a stepping stone for small systems).

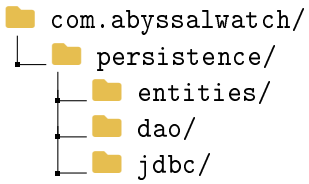

Figure 1.3: Entity types nested under the data access layer, closest to the database.

1.1.7 Classic Build Order

When a team adds a feature in this style, work usually flows **bottom-up**:[6]

1. design tables and build the data access layer (mapping, DAOs),
2. write business services that use those data shapes,
3. expose services through UI or API controllers.

1.2 Step 1: Database and Data Access Layer

We start with the database because the schema is the blueprint. For `Confirm First Kaiju Detection` we need at least a `breach` table and a `kaiju` table with a foreign key to the origin breach (`INV-04`). For `View Mission Kaiju Briefing` (Section 1.8) we also need a `mission` table linked to the tracked Kaiju.

1.2.1 Schema and Entity Mapping

A simplified relational sketch:

- `breach(id, ...)`
- `kaiju(id, codename, classification, origin_breach_id, mission_id, ...)` with `origin_breach_id` referencing `breach(id)` and `mission_id` referencing `mission(id)` when assigned
- `mission(id, status, ...)` for briefing state (for example `Briefed`, `Deployed`)

We map these tables to entity classes in `persistence.entities`; Code snippet 1.1 shows the shape for `kaiju`. Field names mirror the columns in the `CREATE TABLE` scripts you write. Close alignment between tables and entity classes is a **feature** in data-centric systems: it keeps transactional CRUD predictable.[6]

```
// persistence.entities - table-shaped entity (no ORM)
public class KaijuEntity
{
    private String id;
    private String codename;
    private KaijuClassification classification;
    private String originBreachId;
    // constructors, getters, setters omitted
}
```

Code snippet 1.1: `KaijuEntity` table-shaped persistence entity

1.2.2 DAOs and Unit of Work

The data access layer exposes DAOs defined from storage needs. The business layer depends on these abstractions; SQL and JDBC details stay in `persistence.jdbc`.
Contracts are often *provider-shaped*: interfaces mirror what storage already offers (`findById`, `save`) because the data access layer defines them from tables first. That is mockable and effective for data-centric work.
A table-shaped method fits when the schema supports it: `KaijuDao.findByMissionId(...)` in Code snippet 1.2 because `kaiju.mission_id` links a Kaiju to a mission. A use-case-shaped need such as *find all sensors within radius of a breach* does not map cleanly to one table method—the DAO starts to fight the vocabulary the use case needs. That friction is a signal to evolve toward core-owned ports (Chapter **??**), not a reason to abandon DAOs for straightforward CRUD.

```
public interface BreachDao
{

    Optional<BreachEntity> findById(BreachId breachId);

}

public interface KaijuDao
{

    void save(KaijuEntity kaiju);

    Optional<KaijuEntity> findByMissionId(MissionId missionId);

}

public interface UnitOfWork
{

    <T> T execute(Supplier<T> action);

}
```

Code snippet 1.2: `BreachDao`, `KaijuDao`, and `UnitOfWork` contracts

The JDBC implementations below follow the same pattern: obtain a `Connection` from `ConnectionProvider.current()`, run parameterized SQL, and translate `SQLException` into `PersistenceException`.

```
public final class JdbcBreachDao implements BreachDao
{
    private final ConnectionProvider connections;

    @Override
    public Optional<BreachEntity> findById(BreachId breachId)
    {
        String sql = "SELECT id FROM breach WHERE id = ?";
        Connection conn = connections.current();
        try (PreparedStatement ps = conn.prepareStatement(sql))
        {
            ps.setString(1, breachId.value());
            try (ResultSet rs = ps.executeQuery())
            {
                if (!rs.next())
                {
                    return Optional.empty();
                }
                return Optional.of(
                    new BreachEntity(rs.getString("id"))
                );
            }
        }
        catch (SQLException ex)
        {
            throw new PersistenceException(
                "Failed to load breach",
                ex
            );
        }
    }
}
```

Code snippet 1.3: `JdbcBreachDao.findById`

`JdbcBreachDao.findById` is a read-only lookup: a single `SELECT` with one bind parameter, returning `Optional.empty()` when no row exists.

```
public final class JdbcKaijuDao implements KaijuDao
{
    private final ConnectionProvider connections;

    @Override
    public void save(KaijuEntity kaiju)
    {
        String sql = """
            INSERT INTO kaiju (
                id, codename, classification, origin_breach_id
            )
            VALUES (?, ?, ?, ?)
            """;
        Connection conn = connections.current();
        // Do not close conn: UnitOfWork owns the lifecycle.
        try (PreparedStatement ps = conn.prepareStatement(sql))
        {
            ps.setString(1, kaiju.getId());
            ps.setString(2, kaiju.getCodename());
            ps.setString(
                3,
                kaiju.getClassification().name()
            );
            ps.setString(4, kaiju.getOriginBreachId());
            ps.executeUpdate();
        }
        catch (SQLException ex)
        {
            throw new PersistenceException(
                "Failed to save kaiju",
                ex
            );
        }
    }
}
```

Code snippet 1.4: `JdbcKaijuDao.save`

`JdbcKaijuDao.save` performs an `INSERT` whose columns mirror `KaijuEntity`. The comment on `connections.current()` matters: the DAO must not close the connection while `UnitOfWork` still owns the transaction.

```java
public final class JdbcUnitOfWork implements UnitOfWork
{
    private final ConnectionProvider connections;

    @Override
    public <T> T execute(Supplier<T> action)
    {
        try (Connection conn = connections.open())
        {
            conn.setAutoCommit(false);
            try
            {
                connections.bind(conn);
                T result = action.get();
                conn.commit();
                return result;
            }
            catch (RuntimeException ex)
            {
                conn.rollback();
                throw ex;
            }
            finally
            {
                connections.unbind();
            }
        }
        catch (SQLException ex)
        {
            throw new PersistenceException(
                "Transaction failed",
                ex
            );
        }
    }
}
```

Code snippet 1.5: `JdbcUnitOfWork.execute`

`JdbcUnitOfWork.execute` opens the connection, disables auto-commit, binds it for nested DAO calls, and commits or rolls back on success or failure.

```
public interface ConnectionProvider
{

    Connection open();

    void bind(Connection connection);

    Connection current();

    void unbind();

}
```

Code snippet 1.6: `ConnectionProvider`

In Code snippet 1.6, `ConnectionProvider` supplies the JDBC `Connection` for the current unit of work: `open` and `bind` for `JdbcUnitOfWork`, `current` for DAOs in the same transaction. DAO implementations must not close a connection obtained from `current()` while the unit of work is still active.

1.2.3 Database Constraints and Business Rules

Use the database for **structural integrity** and the business layer for **policy**:

- **Database:** foreign keys (for example `INV-04` via `origin_breach_id`), `NOT NULL`, unique indexes—the safety net that still holds under races and alternate callers.
- **Business layer:** mission state gates, classification rules, and user-facing validation messages—the place that expresses intent.
- **Avoid:** triggers or stored procedures that encode policy (Section 1.1.3).

For `Confirm First Kaiju Detection`, the service in Code snippet 1.7 checks that the origin breach exists before insert; the foreign key on `origin_breach_id` still protects integrity if two requests race. Prefer friendly errors from the service; let the database enforce what must never be violated.

1.3 Step 2: Business Logic Layer

With data structures in place, we write business services that load and save through the data access layer and enforce procedural rules. For `Confirm First Kaiju Detection`, the service validates that the origin breach exists, constructs a new Kaiju record shape, and persists it inside a transaction.
The business layer knows entity shapes because the data access layer exposed them (Code snippet 1.1)—not because we modeled the use case in a vacuum first.

1.3.1 Transaction Script Services

Table-shaped types in `persistence.entities` are intentionally **anemic** in this style—data containers with little or no behavior. Rules and sequencing live in services (and helpers such as `factory` or `assembler`), not in entity methods. That differs from Fowler's domain-layer variation in Section 1.1.5, where behavior may live in a model that is not tied one-to-one to tables.

1.3.2 Business Service Example in Java

```java
public final class ConfirmKaijuDetectionService
{
    private final BreachDao breachDao;
    private final KaijuDao kaijuDao;
    private final UnitOfWork unitOfWork;

    public KaijuId confirmDetection(
        String codename,
        KaijuClassification classification,
        BreachId originBreachId
    )
    {
        return unitOfWork.execute(() ->
        {
            breachDao.findById(originBreachId)
                .orElseThrow();

            KaijuEntity kaiju = new KaijuEntity();
            kaiju.setId(KaijuId.generate().value());
            kaiju.setCodename(codename);
            kaiju.setClassification(classification);
            kaiju.setOriginBreachId(originBreachId.value());

            kaijuDao.save(kaiju);
            return new KaijuId(kaiju.getId());
        });
    }
}
```

Code snippet 1.7: `ConfirmKaijuDetectionService` transaction script

Code snippet 1.7 follows Fowler's **Transaction Script** pattern: a procedural script over the DAOs in Code snippet 1.2, with one use case per method.[2]

IN-REF: add reference to Transaction Script pattern chapter when written

1.4 Step 3: Presentation Layer

Diagram placeholder

KaijuController → ConfirmKaijuDetectionService → DAOs → database

DIAGRAM: write use case flow: presentation controller to ConfirmKaijuDetectionService to DAOs to database

Figure 1.4: `Confirm First Kaiju Detection` write flow through the layers (diagram to be added).

Finally we expose business services to users. Figure 1.4 summarizes the write path for `Confirm First Kaiju Detection`. The presentation layer stays thin: parse input, call the business service, format the response. It must not embed business rules or call DAOs directly.

The examples use **Spring MVC** annotations (`@RestController`, `@PostMapping`) as one common way to expose HTTP in Java. The architectural rule is thin adapters that delegate to services—not Spring itself. The same structure applies with other web frameworks or with non-HTTP entry points (Section 1.4.2).
Typed request and response records can live in `presentation.web` at first, or in a dedicated `dtos` module when API contracts need stronger isolation (Section 1.6).

1.4.1 Presentation Mapping Example in Java

Code snippet 1.8 shows a thin REST adapter: it maps HTTP input to service parameters and delegates to Code snippet 1.7.

```
public record ConfirmKaijuDetectionRequest(
    String codename,
    KaijuClassification classification,
    BreachId originBreachId
) {}

public record ConfirmKaijuDetectionResponse(String kaijuId) {}

@RestController
@RequestMapping("/kaiju-detections")
public class KaijuController
{
    private final ConfirmKaijuDetectionService service;

    @PostMapping
    public ResponseEntity<ConfirmKaijuDetectionResponse> confirm(
        @RequestBody ConfirmKaijuDetectionRequest request
    )
    {
        KaijuId id = service.confirmDetection(
            request.codename(),
            request.classification(),
            request.originBreachId()
        );

        return ResponseEntity.accepted()
            .body(new ConfirmKaijuDetectionResponse(id.value()));
    }
}
```

Code snippet 1.8: `KaijuController` REST presentation adapter

1.4.2 Other Driving Entry Points

Presentation is any **driving** entry point that translates outside interaction into calls the business layer understands. In this chapter we use a REST controller (Code snippet 1.8) because three-layered systems in the course are usually exposed through HTTP. The same business service (Code snippet 1.7) could equally be invoked from a CLI handler, a scheduled job, or a message consumer: keep the adapter thin, delegate to the service, and do not call DAOs directly on writes. Business rules stay in `application`; only the transport changes.

1.4.3 Validation, Errors, and Leaky Boundaries

Each layer has a distinct job when input fails or storage rejects a change:

- **Presentation** parses and validates *shape*: required fields, formats, and ranges suitable for HTTP or UI. It maps outcomes to status codes or messages the client sees. It does not decide business policy.
- **Business logic** enforces *policy*: for example that the origin breach exists before insert (Section 1.2.3). Prefer clear application exceptions (`UnknownBreachException`) over generic failures.
- **Data access** runs SQL and translates `SQLException` into persistence-layer failures. Map those to business-meaningful errors at the boundary before they reach presentation, rather than leaking raw SQL state upward.

Leaky abstractions appear when layers share the wrong types: `SQLException` or `ResultSet` in a controller, HTTP request types in a service, or persistence entities returned directly from REST without a DTO evolution path (Section 1.6). The `orElseThrow()` calls in Code snippets 1.7 and 1.8 are placeholders; production code should throw domain- or application-level errors the presentation layer can translate consistently.

1.5 When the Business Layer Grows

A small system can live in a flat `application.service` package. `Confirm First Kaiju Detection` fits that shape. When more use cases arrive, two problems appear: **service-to-service cycles** and **responsibilities that do not fit a single service class**.

1.5.1 Breaking Cycles Between Services

A textbook pattern outside our case study: `CustomerService` and `OrderService` each grow features that need the other (`customer with order history` vs. `order with customer details`). Peer service calls create a circular dependency.
The same pattern appears in `Abyssal Watch`: `MissionService` (briefing with Kaiju summary) and `KaijuTrackingService` (track with mission context) may tempt each other into cross-calls. Extract shared linkage logic into `application.shared`—for example `MissionKaijuContext`—that both services use. `shared` must not depend on `service`, and services must not depend on each other.

1.5.2 Sibling Helpers Inside the Business Layer

When logic is real business concern but clutters a service, use sibling packages by *kind* of work: `factory` (construction and invariants), `assembler` (combining data for a use case or view), and `shared` (cross-use-case linkage). A `KaijuFactory` can enforce `INV-04`; `MissionBriefingAssembler` can build the briefing view in Code snippet 1.11; triangulation fusion for `SCN-03` may live in another `assembler`. Services still own transaction boundaries and DAO calls.
Optional evolution: when a service method such as `confirmDetection` accumulates many parameters, Fowler's *Introduce Parameter Object* refactor groups them into one type.[3] That is an alternative to growing a long parameter list; defer it until the signature is genuinely hard to read.
Do not confuse `application.shared` with `persistence.entities`: the latter holds table-shaped persistence entities; `shared` holds cross-use-case business logic. Do not confuse either with `dtos` (Section 1.6), which are presentation-facing contracts.

IN-REF: add reference to Object Parameter Pattern chapter when written

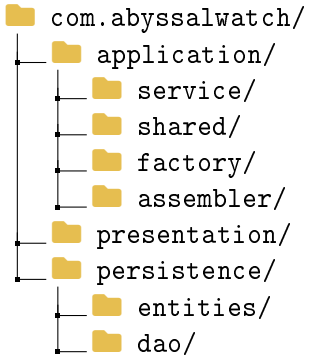

Figure 1.5: Business layer sub-packages when the flat `service` package is no longer enough.

Introduce sub-packages only when the flat layout is hard to navigate—not on day one.

1.5.3 Dependencies Inside the Application Layer

Inside `application`, dependencies should form a directed acyclic graph:

- `service` may depend on `shared`, `factory`, and `assembler`,
- `shared`, `factory`, and `assembler` must not depend on `service`,
- services must not depend on each other—extract shared logic (for example `MissionKaijuContext`) instead.

Keep types in the right bucket:

- `persistence.entities` — table-shaped persistence entities,
- `application.shared` — cross-use-case business logic,
- `dtos` — presentation-facing contracts (Section 1.6).

DIAGRAM: application sub-package DAG: service to shared, factory, assembler; no service-to-service edges

Diagram placeholder

service → shared, factory, assembler; no service-to-service edges

Figure 1.6: Directed dependencies inside the `application` layer (diagram to be added).

1.5.4 Layer Packages vs Feature Packages

The layouts in this chapter use **layer packages**—`presentation`, `application`, `persistence`—because they match the bottom-up, schema-first workflow students follow: design tables, write JDBC DAOs (Code snippets 1.3–1.6), add services, then controllers. Layer placement is easy to teach and review.

Feature packages (or vertical slices) group everything for one capability together, for example `kaiju.detection` containing controller, service, and DAO for that flow. That helps when one

feature changes often and the team wants related code in one place. The trade-off is that technical concerns scatter: SQL, HTTP mapping, and policy for the same story live in different roots unless you discipline sub-structure.
A practical hybrid keeps layer roots but adds feature subfolders under `application.service` when a domain area grows (for example `service.kaiju`, `service.mission`). Hexagonal architecture discusses feature-folder trade-offs in more depth (Chapter **??**); here layer packages remain the default for data-centric three-layer systems.

1.6 Evolution: DTO Module

A dedicated `dtos` module is **not** required for the classic baseline. Add it when HTTP or UI contracts must stay stable while entity or service signatures change—versioning, hiding persistence fields, or shaping responses per use case.
Mapping adds code, but it decouples presentation from table-shaped types exposed by the business and data layers.

1.6.1 DTO Example in Java

When API contracts must stay stable while service or entity signatures change, move types such as `ConfirmKaijuDetectionRequest` and `ConfirmKaijuDetectionResponse` from `presentation.web` into a dedicated `dtos` module (see Code snippet 1.8 for the baseline shape).

1.6.2 Who Maps What

Mapping belongs at the boundary that introduces a new representation:

- **Presentation** maps HTTP or UI input to service parameters or `dtos`, and maps results to responses (Code snippet 1.8).
- **Application** maps entity shapes to use-case inputs; `assembler` helpers compose read models when using business-layer assembly.
- **Data access** maps entities to the database with SQL; read projections for view assembly live here in Approach A (Section 1.8).

Avoid passing the same shape through every layer by habit. Add mapping when a transport or storage technology forces a distinct type.

1.7 Recommended Package and Module Layout

DIAGRAM: module dependency arrows: presentation to application to persistence; config wires implementations

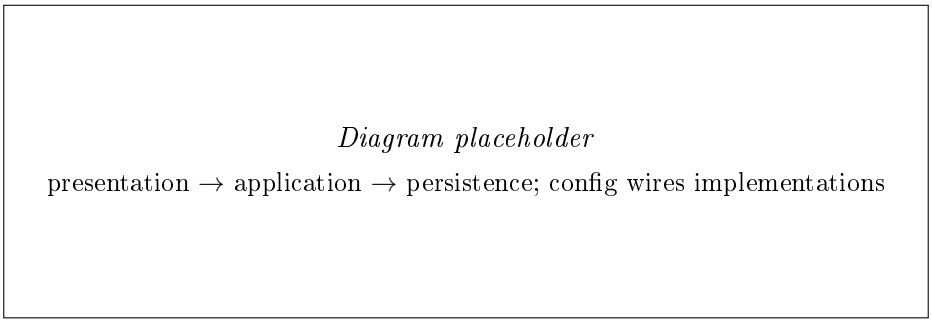

Figure 1.7: Module dependency direction and configuration wiring (diagram to be added).

Packages are compile-time folders inside one project—the default for student work in this book. **Modules** (Gradle or Maven) add separate compilation units when the codebase grows; they can enforce that `presentation` does not import `persistence.jdbc` directly. Layer rules are the same whether you use packages only or split modules. Figure 1.7 shows the intended dependency arrows between modules.

IN-REF: float these package diagrams to the right, wrap text on the left, like in hexagonal chapter

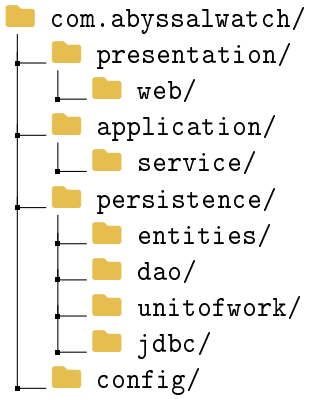

Figure 1.8: Baseline package layout for a data-centric three-layer monolith.

Figure 1.8 shows the baseline data-centric package layout, you will notice the top level packages are the layers of the architecture: presentation, application, and persistence.
When the business layer grows, add sub-packages under `application` as in Figure 1.5. When API contracts need stability, add a root `dtos` module (Figure 1.9):

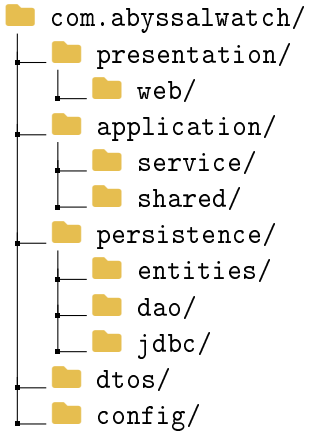

Figure 1.9: Package layout with a separate `dtos` module.

If requirements expand beyond database access, rename the persistence layer to `infrastructure` to better reflect its more adverse role (Figure 1.10):

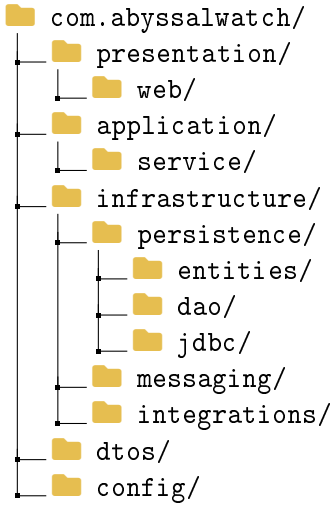

Figure 1.10: Package layout with `infrastructure` outer ring.

Dependency rules:

- `presentation` depends on `application` (and `dtos` when introduced),
- `application` depends on `persistence` DAOs and `UnitOfWork`,
- `persistence` depends on `persistence.entities` and storage technology,
- `presentation` must not call `persistence` directly,
- `config` wires implementations at startup.

1.8 Assembling Data for Views

Read screens often need joined or aggregated data, not a single entity row. We use `View Mission Kaiju Briefing` as the running read example: for a `MissionId`, the UI shows mission status, linked Kaiju codename and classification, and the origin breach label—data drawn from `mission`, `kaiju`, and `breach`.

Both assembly strategies below produce the same `MissionKaijuBriefingView` (Code snippet 1.9); they differ in *where* joins and composition happen.[6] Figure 1.11 contrasts the two paths.

DIAGRAM: Mission Kaiju Briefing: DAL join query path vs BLL multi-DAO assembler path

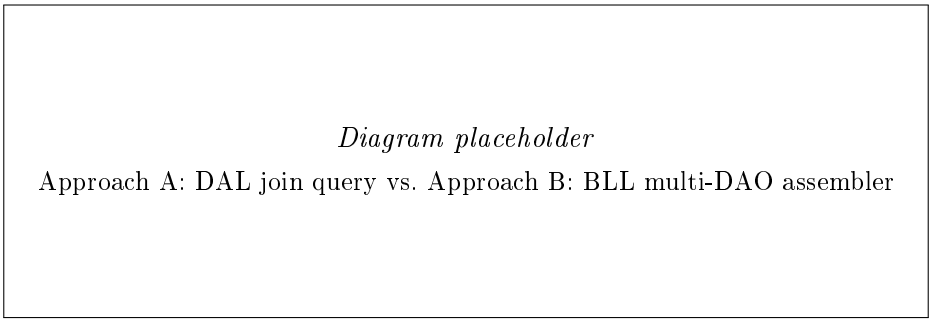

Figure 1.11: `View Mission Kaiju Briefing`: data-access join path vs. business-layer assembly path (diagram to be added).

1.8.1 Approach A: Data-Access-Level Assembly

In **Approach A**, Code snippet 1.9 defines the view record and a DAO method that returns a projection, often via a single join query:

```
public record MissionKaijuBriefingView(
    String missionId,
    String missionStatus,
    String kaijuCodename,
    KaijuClassification kaijuClassification,
    String originBreachLabel
) {}

public interface MissionBriefingDao
{

    Optional<MissionKaijuBriefingView> findBriefing(
        MissionId missionId
    );

}
```

Code snippet 1.9: `MissionKaijuBriefingView` and `MissionBriefingDao`

A `JdbcMissionBriefingDao` implements `MissionBriefingDao` from Code snippet 1.9 with a single `JOIN` across `mission`, `kaiju`, and `breach`—the SQL lives in `persistence.jdbc`, not in the business layer.
Presentation may call a thin `MissionBriefingQueryService` that delegates to the DAO, or—when the read is purely presentational—use a relaxed path (Section 1.1.4) as in Code snippet 1.10.

```java
@RestController
@RequestMapping("/missions")
public class MissionBriefingReadController
{
    private final MissionBriefingDao briefingDao;

    @GetMapping("/{missionId}/briefing")
    public MissionKaijuBriefingView briefing(
        @PathVariable String missionId
    )
    {
        return briefingDao.findBriefing(new MissionId(missionId))
            .orElseThrow();
    }
}
```

Code snippet 1.10: `MissionBriefingReadController` data-access read path

1.8.2 Approach B: Business-Layer Assembly

In **Approach B**, the business layer loads persistence entities through several DAOs and composes the same view with an `assembler`, keeping policy explicit:

```
public final class GetMissionBriefingService
{
    private final MissionDao missionDao;
    private final KaijuDao kaijuDao;
    private final BreachDao breachDao;
    private final MissionBriefingAssembler assembler;

    public MissionKaijuBriefingView getBriefing(
        MissionId missionId
    )
    {
        MissionEntity mission = missionDao.findById(missionId)
            .orElseThrow();

        if (!Set.of("Briefed", "Deployed")
            .contains(mission.getStatus()))
        {
            throw new IllegalStateException(
                "Briefing not available"
            );
        }

        KaijuEntity kaiju = kaijuDao.findByMissionId(missionId)
            .orElseThrow();
        BreachEntity breach = breachDao.findById(
            new BreachId(kaiju.getOriginBreachId())
        ).orElseThrow();

        return assembler.toView(mission, kaiju, breach);
    }
}
```

Code snippet 1.11: `GetMissionBriefingService` business-layer assembly

Code snippet 1.11 loads entities through the DAOs in Code snippet 1.2 and applies briefing policy before assembly. The same HTTP resource can call the service instead of the DAO directly, as in Code snippet 1.12; the controller only maps the request and response.

```java
@RestController
@RequestMapping("/missions")
public class MissionBriefingController
{
    private final GetMissionBriefingService briefingService;

    @GetMapping("/{missionId}/briefing")
    public MissionKaijuBriefingView briefing(
        @PathVariable String missionId
    )
    {
        return briefingService.getBriefing(
            new MissionId(missionId)
        );
    }
}
```

Code snippet 1.12: `MissionBriefingController` business-layer read path

1.8.3 Same View, Different Trade-offs

Approach A (Code snippets 1.9 and 1.10) suits stable briefing screens when query efficiency matters: fewer round-trips, database-optimized joins, slimmer business code for read-only flows. Costs: data access absorbs view shapes; DAO APIs multiply as screens evolve.
Approach B (Code snippets 1.11 and 1.12) suits the same view when policy and reuse matter: briefing rules (for example allowed mission states) stay in the service; data access stays entity-oriented. Costs: more DAO calls unless carefully batched; more coordination code—mitigate with `MissionBriefingAssembler`.
For `SCN-03 Multi-Scout Triangulation`, non-trivial fusion rules often push assembly into the business layer with a dedicated `assembler`, even when some sensor reads still come from specialized DAO queries.

1.8.4 SQL Pitfalls When the Business Layer Orchestrates Reads

Approach B in Code snippet 1.11 loads `mission`, `kaiju`, and `breach` through separate DAO calls. With JDBC that usually means **multiple round-trips** to the database—the same class of cost people later call N+1 when using ORMs. For hot read screens, prefer Approach A's single join behind Code snippet 1.9 unless policy truly belongs in the service.
Connection and transaction scope: one `Connection` per unit of work (Code snippet 1.5). DAOs must use the connection bound for the current transaction; they must not open silent nested transactions or new connections per call inside the same use case.
Keep SQL behind DAOs: services receive entities or view records from DAO interfaces—not ad-hoc SQL strings. If a course later adopts JPA or Hibernate, lazy-loading and session-boundary mistakes recreate similar read and transaction bugs; this chapter stays with explicit JDBC so those boundaries remain visible.

1.8.5 CQRS Perspective for Read Models

When read complexity dominates, a CQRS-inspired split can separate write entities from read models. That improves read performance but adds maintenance and possible eventual consistency. Treat it as optional evolution, not a day-one default.

1.8.6 Decision Guidance

As a guideline:

- prefer data-access-level assembly when query efficiency matters and view shapes are relatively stable,
- prefer business-layer assembly when policy and cross-channel reuse matter more,
- evaluate CQRS when read scale and complexity dominate.

1.9 Testing Strategy Across Layers

Testing follows the bottom-up boundaries:

- **Data access:** integration tests against a real database—verify SQL, entity mapping, and transactions,
- **Business logic:** tests with fake DAOs that implement the same interfaces as Code snippet 1.2,
- **End-to-end:** a small set of full request-to-response flows through presentation.

For `Confirm First Kaiju Detection`, a focused business-layer test constructs the service from Code snippet 1.7 with `FakeBreachDao`, `FakeKaijuDao`, and an `ImmediateUnitOfWork`, then asserts the service saved the expected `KaijuId`. The fakes implement the same DAO interfaces as Code snippet 1.2—they are not contracts invented by the business core in isolation.

1.10 When Persistence Becomes Infrastructure

1.10.1 Case Trigger from Abyssal Watch

`Confirm First Kaiju Detection` may need to publish a mission alert, not only insert a row. Messaging and integrations belong in the same *outer* ring as storage, still below business logic.

1.10.2 Why Persistence Can Be Too Narrow

The name `persistence` fits when the outer layer is mostly database work. When messaging, APIs, files, or telemetry stabilize alongside storage, `infrastructure` is a clearer name. Dependency direction is unchanged: `presentation -> application -> infrastructure`.

1.10.3 Recommended Evolution Path

1. start with `persistence` while the schema-led stack is mostly database access,
2. rename to `infrastructure` when multiple technical capabilities are stable (Figure 1.10),
3. split by capability: `infrastructure.persistence`, `infrastructure.messaging`, and so on.

The data-centric workflow still begins with storage design; additional adapters extend the outer ring, they do not move schema decisions into presentation.

1.10.4 Mission Alert After Detection

`Confirm First Kaiju Detection` may need to notify command workflows, not only insert a row. The **decision to alert** stays in the business service; **delivery** belongs in infrastructure (or `persistence.messaging` until you rename the outer ring).

```
public interface KaijuDetectionAlertPublisher
{

    void publish(KaijuDetectedAlert alert);

}

public final class ConfirmKaijuDetectionService
{
    private final BreachDao breachDao;
    private final KaijuDao kaijuDao;
    private final UnitOfWork unitOfWork;
    private final KaijuDetectionAlertPublisher alertPublisher;

    public KaijuId confirmDetection(
        String codename,
        KaijuClassification classification,
        BreachId originBreachId
    )
    {
        return unitOfWork.execute(() ->
        {
            breachDao.findById(originBreachId)
                .orElseThrow();

            KaijuEntity kaiju = new KaijuEntity();
            // set fields omitted

            kaijuDao.save(kaiju);

            alertPublisher.publish(
                new KaijuDetectedAlert(
                    new KaijuId(kaiju.getId()),
                    originBreachId
                )
            );

            return new KaijuId(kaiju.getId());
        });
    }
}
```

Code snippet 1.13: `KaijuDetectionAlertPublisher` and extended `ConfirmKaijuDetectionService`

Code snippet 1.13 extends Code snippet 1.7 with a narrow `KaijuDetectionAlertPublisher` interface the application depends on, implemented by an adapter in `infrastructure.messaging`. The mechanism can be in-process notification, email, or a message bus—this chapter does not prescribe queues or async plumbing; the adapter is replaceable.

Publishing inside the same unit of work as the database insert can still leave **inconsistent**

outcomes: the row may commit while the alert channel fails afterward, or vice versa if ordering is wrong. Production systems often use an **outbox** table or transactional messaging so the alert is recorded atomically with the write and delivered reliably later. That machinery is out of scope here; the important placement rule remains: the service decides *whether* to alert, infrastructure delivers it. Figure 1.12 shows how the service reaches persistence and the alert adapter.

DIAGRAM: ConfirmKaijuDetectionService to persistence DAOs and to infrastructure.messaging alert adapter

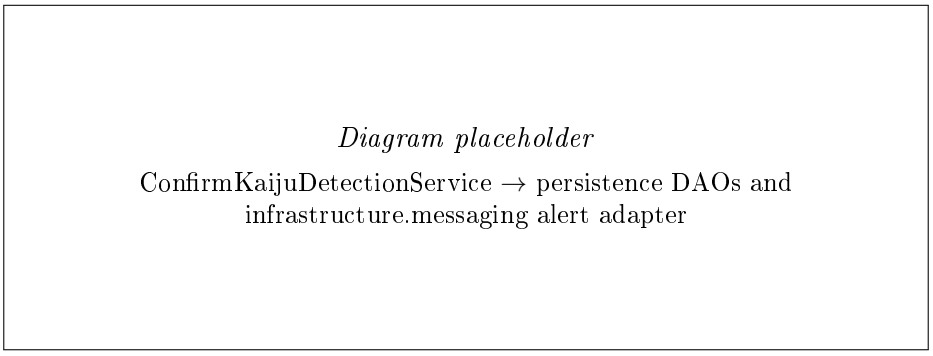

Figure 1.12: Service dependencies on persistence DAOs and an infrastructure alert adapter (diagram to be added).

1.10.5 Trade-offs and Guardrails

Avoid turning `infrastructure` into a dumping ground:

- business decisions stay in `application`,
- organize by capability, not framework package names,
- business layer depends on narrow interfaces per capability,
- no `presentation -> infrastructure` shortcuts,
- do not put alert orchestration in presentation or DAOs.

1.11 Schema Change Ripples

1.11.1 A Worked Example: Renaming a Column

Suppose `origin_breach_id` on `kaiju` is renamed to `breach_of_origin_id`. In a data-centric stack the ripple is predictable:

1. migration script updates the table,
2. Code snippet 1.1 and `KaijuDao` SQL in Code snippet 1.4 change,
3. Code snippet 1.7 and any setter calls adjust,
4. the REST field in Code snippet 1.8 may change,
5. Code snippet 1.9 join projection updates if the briefing query exposed the column.

That coupling is a trade-off, not a surprise. When a public API must stay stable through such refactors, introduce or extend `dtos` (Section 1.6). Keep join-heavy mapping in data access; keep policy in services—not in triggers (Section 1.2.3).

1.12 Practical Trade-offs and Common Mistakes

Strengths:[6]

- fast delivery for CRUD-heavy, transactional applications,
- predictable table-to-object mapping and onboarding,
- clear placement rules for new code,
- Transaction Script services over provider-shaped DAOs are easy to teach and ship.[2]

Costs:

- schema changes propagate through layers (Section 1.11),
- swapping storage technology is harder when business code assumes relational shapes,
- rich behavior can accumulate in oversized service classes without sub-package discipline,
- relaxed reads and DAL view projections can hide coupling to schema and screen shapes.

Common mistakes:

- business rules in controllers or UI components,
- orchestration inside DAOs,
- treating relaxed reads as the default for every query,
- presentation calling data access directly on writes or policy-heavy reads (contrast Code snippets 1.8 and 1.10),
- logic in stored procedures that should live in the business layer,
- exposing raw persistence entities in public APIs without a DTO evolution path,
- alert orchestration in presentation or DAOs instead of the placement in Code snippet 1.13,
- closing the shared `Connection` inside a DAO while a `UnitOfWork` is still active (Code snippets 1.4 and 1.5),
- SQL strings or `PreparedStatement` usage in services instead of DAOs,
- untyped request bodies (for example a generic `Map`) at the presentation boundary without shape validation.

1.12.1 Layered and Hexagonal: Practical Relationship

Data-centric layering and hexagonal architecture address the same problem—separating behavior from technology—with different anchors. Classic three-layer starts from the **database schema** and builds upward; hexagonal starts from **use-case behavior** and plugs technology in through ports. Neither is universally "correct."

When integration churn, testing friction, or the need for core-owned contracts grows, teams can evolve selected areas toward ports and adapters (Chapter **??**). That is a refinement path, not an admission that layering was wrong for systems that genuinely center on accurate record capture and storage.

1.12.2 Closing Thoughts

This chapter taught **data-centric three-layered architecture** inside a logical monolith: the relational schema is the anchor, and software grows upward in predictable layers. The journey was intentionally bottom-up: design tables and write SQL, implement JDBC DAOs and persistence entities (Code snippets 1.1–1.6), add Transaction Script services (Code snippet 1.7) with explicit transaction boundaries, expose thin REST adapters (Code snippet 1.8), then extend the shape when complexity appears—shared helpers and assemblers, optional `dtos` (Figure 1.9), infrastructure for alerts (Code snippet 1.13), read assembly (Code snippets 1.9–1.12), and controlled relaxed reads where policy does not apply. Sections on DTO modules, infrastructure renaming, and advanced read strategies are **evolution** topics, not day-one requirements.
For your course work, hand-written SQL stays in `persistence`; layers exist so that SQL, HTTP details, and business policy do not collapse into one place. When provider-shaped DAOs, testing friction, or integration churn grow, hexagonal architecture offers a behavior-first refinement (Section 1.12.1)—not because layering failed, but because the problem outgrew schema-led defaults. For CRUD-heavy, transactional systems that genuinely center on accurate record capture, the classic three layers remain a strong, teachable default.

Chapter 2

Functional Core, Imperative Shell

Functional Core, Imperative Shell (FCIS) is an architectural discipline for separating *what to decide* from *how to reach the outside world* [1, 8].
While I don't get the impression that this pattern is widely used, and I don't have much experience with it, I personally like it for its simplicity and its ability to isolate the pure logic from the outside world, which makes it *very* easy to test. Your logic code can also be broken down into small functions, generally making your code easier to understand and maintain.
The **functional core** holds business rules, calculations, and data transformations as pure, deterministic code. The **imperative shell** performs side effects: receiving input from the outside world (presentation layer, web api, cli, gui, etc.),loading data, calling external systems, saving results, and publishing messages. All these activities are interacting with the outside world.
Gary Bernhardt popularised the idea in his *Boundaries* talk; since then it has become a practical way to keep complex logic testable in mainstream object-oriented languages.[5, 9, 4]
FCIS is not necessarily a replacement for layered or hexagonal architecture. It can be incorporated into these architectures, or it can be used standalone. It is a cross-cutting refinement you can apply *inside* a service class, a package, or a module. A hexagonal use case can still have ports and adapters; FCIS tells you where the pure calculation lives and where orchestration belongs. Any calculation-heavy service—whether it owns transactions, calls repositories, or publishes events—benefits from the same split between pure rules and imperative wiring.
I will, however, try to minimize references to hexagonal architecture, and other architectural styles, to keep the focus on the FCIS pattern itself. You may then extract ideas and apply them to other architectures, if you like.
In this chapter we use the `Abyssal Watch` case study with two running examples:

- **SCN-03 — Triangulate Kaiju Position:** fuse multiple sensor readings into an estimated area (geometry and data fusion),
- **SCN-04 — Reconcile Mission Telemetry:** merge partially uplinked observations when a Scout Vessel docks (sync and deduplication).

The first example revisits triangulation from Chapter **??** through an FCIS lens: we extract pure fusion math from a service you have already seen. The second example introduces a **new** retrieval-time problem—intermittent uplink during mission closure—that has not appeared in prior chapters.
Both examples highlight where functional programming ideas shine: multi-step transformations where you want fast, deterministic unit tests without mocks or databases.
We begin with a brief introduction to functional programming concepts, then define the FCIS pattern, the *imperative sandwich*, and work through both examples end to end.

2.1 Functional Programming Concepts

Before we apply FCIS, we need a shared vocabulary. The terms below appear throughout this chapter and in much of the functional-programming literature.[8]

2.1.1 Functional Programming

Functional programming treats computation primarily as the evaluation of functions over values. Programs are built by composing functions that take inputs and return outputs, rather than by issuing sequences of commands that mutate shared state. In practice, functional style emphasises:

- functions as first-class values (they can be passed, returned, and stored),
- expressions over statements where possible,
- immutable data structures,
- and avoiding shared mutable state across concurrent work.

Languages such as Haskell and Clojure are fully functional; Java, C#, and Python can adopt functional *discipline* in selected areas without becoming functional languages wholesale. FCIS is exactly that: apply functional discipline to the parts of your system that are really about calculation and decision-making.

2.1.2 Imperative Programming

Imperative programming expresses computation as a sequence of commands that change program state. "Do this, then that, then update this variable" is the natural shape. Most business applications are imperative at the edges: read a row from the database, send an HTTP response, write a log line. That is not a flaw—the outside world is stateful and effectful. FCIS does not try to eliminate imperative code; it *contains* it in a thin shell so the core stays easy to reason about.

2.1.3 Honest Functions

An **honest function** is one whose signature tells the truth about what it needs and what it does.[8] If a method is named `calculateScore(observations)` but secretly reads from a database, consults a global configuration object, or logs to the console, it is *dishonest*: callers cannot understand or test it from the parameters alone. Dishonest functions hide dependencies, which forces integration tests, mocks, and surprises in production.
Honest functions take everything they need as explicit arguments and return explicit results. That honesty is what makes the functional core testable: the test passes data in and asserts on data out, with no hidden collaborators.

2.1.4 Pure and Impure Functions

A **pure function** is deterministic and free of side effects: given the same inputs, it always produces the same outputs, and it does not observe or change anything outside its local scope. An **impure function** does anything else—reads the clock, queries a database, mutates a field on a shared object, or writes a file.
Pure functions are the building blocks of the functional core. Impure functions belong in the imperative shell. The boundary is pragmatic: a method that only transforms in-memory data structures is pure; a method that opens a JDBC connection is not.

Code snippet 2.1 shows an impure method that mixes a database read with a simple calculation. Code snippet 2.2 extracts the calculation into a pure function; the shell (not shown here) would load observations and pass them in.

```
// Impure: loads data inside the "calculation"
public double summarizeObservations(MissionId missionId)
{
    List<ObservationEntity> rows =
        observationDao.findByMission(missionId);

    double total = 0.0;
    for (ObservationEntity row : rows)
    {
        total += row.getSignalStrength();
    }

    return total;
}
```

Code snippet 2.1: Impure summary: database access mixed with calculation

```
// Pure: same inputs always yield the same total
public static double summarizeSignalStrength(
    List<ObservationSnapshot> observations
)
{
    return observations.stream()
        .mapToDouble(ObservationSnapshot::signalStrength)
        .sum();
}
```

Code snippet 2.2: Pure summary: explicit inputs, explicit result

2.1.5 Immutable and Mutable Data

Mutable data can be changed after creation; **immutable data** cannot. When several threads or several layers share mutable objects, reasoning about who changed what becomes expensive. Pure functions work best with immutable inputs: they return new values instead of altering arguments in place.

In Java, `record` types are a practical default for values crossing the core boundary. Prefer `List.copyOf` or unmodifiable lists when exposing collections from the core, and treat any object passed *into* the core as read-only unless you have a compelling reason otherwise. The shell may still use mutable JDBC entities internally; map them to immutable snapshots before calling the core.

2.1.6 Side Effects

A **side effect** is any interaction with the outside world or any observable change beyond returning a value. Common side effects include:

- database reads and writes,

- network and message-broker calls,
- file system access,
- logging and metrics,
- mutation of shared or global state,
- reading non-deterministic sources such as the system clock or a random number generator.

Side effects are necessary in real systems. FCIS confines them to the imperative shell so the functional core stays a pipeline of honest, pure transformations.

2.1.7 Applying These Ideas in Java

Java is not a functional language, but FCIS maps cleanly onto familiar tools:

- **Core package:** `static` pure methods or small `final` classes with no infrastructure imports. No Spring, JDBC, JPA, or HTTP types.
- **Value types:** Java `record` for inputs and outputs (`ObservationSnapshot`, `ReconciledTelemetry`, `AnnularSector`).
- **Shell package:** services that inject DAOs, repositories, and messaging adapters; they load data, call the core, then persist or publish results.
- **Immutability:** `final` fields on services; defensive copies at the core boundary.
- **Optional:** express absence without `null`; the core returns data, the shell decides whether to throw or map to HTTP errors.
- **Push rule:** when you write an `if` in the shell, ask whether the condition could be computed by a pure function instead.[5]

The rest of this chapter applies these rules to two Abyssal Watch use cases.

2.2 Why We Use Functional Core, Imperative Shell

2.2.1 Testability Without Mocks

The main payoff is **fast, focused unit tests**. When business rules live in pure functions, tests pass plain data structures in and assert on returned values. You do not need fake databases, stubbed HTTP clients, or mocked repositories to verify scoring logic or geometric fusion. Integration tests still matter for the shell—wiring, SQL, and transaction boundaries—but the expensive, branching logic is already proven in isolation.

2.2.2 Clear Separation of Concerns

FCIS forces a repeatable shape: **read** data, **compute** a result, **write** or publish. That read-compute-write rhythm matches how we describe use cases to non-developers ("load the readings, figure out where the Kaiju is, store the estimate") and matches how we draw sequence diagrams. When calculation is buried inside DAO callbacks or controller actions, the story is harder to follow and harder to change.

2.2.3 Safer Concurrency

Pure functions do not share mutable state. If the core only transforms immutable values, you can call it from multiple threads without locks, and you can parallelise independent calculations when needed. The shell still coordinates I/O and transactions; the core stays embarrassingly parallel friendly.

2.2.4 Relationship to Other Architectural Styles

FCIS complements other styles taught in this book:

- In **three-layered architecture** (Chapter 1), services such as `ConfirmKaijuDetectionService` often grow procedural scripts. FCIS splits the script into pure rules and imperative sequencing.
- In **hexagonal architecture** (Chapter **??**), ports and adapters define *where* technology plugs in; FCIS defines *what* inside the use case must stay pure. A thin `TriangulateKaijuPositionService` can still depend on `SensorRepository` and `SensorDriver` ports while delegating geometry to `KaijuPositionEstimator`.
- FCIS does not require extra port interfaces. A minimal adoption is one extracted pure class and a slimmer service method.

2.3 The Imperative Sandwich

The **imperative sandwich** (also called the **functional core sandwich**) is the request-level flow that FCIS encourages:[8, 5]

1. **Read (imperative):** the shell fetches data from databases, APIs, sensors, or user input.
2. **Compute (functional):** the shell passes plain data into the core; the core returns a new value with no side effects.
3. **Write (imperative):** the shell persists results, sends responses, publishes events, or triggers hardware.

The imperative layers are the bread; the pure core is the filling. Keep the bread thin and the filling rich.

DIAGRAM: imperative sandwich: read I/O, pure functional core, write I/O

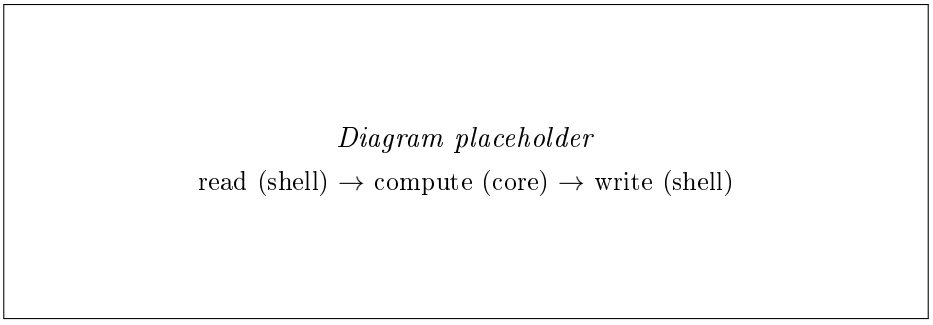

Figure 2.1: The imperative sandwich: side effects on the outside, pure logic in the middle (diagram to be added).

Figure 2.1 summarises the pattern. Compare that to a **fat service** where every method interleaves SQL, conditionals, and mapping—the same use case works, but tests must mimic the world to reach the logic.

DIAGRAM: fat service vs imperative sandwich side by side

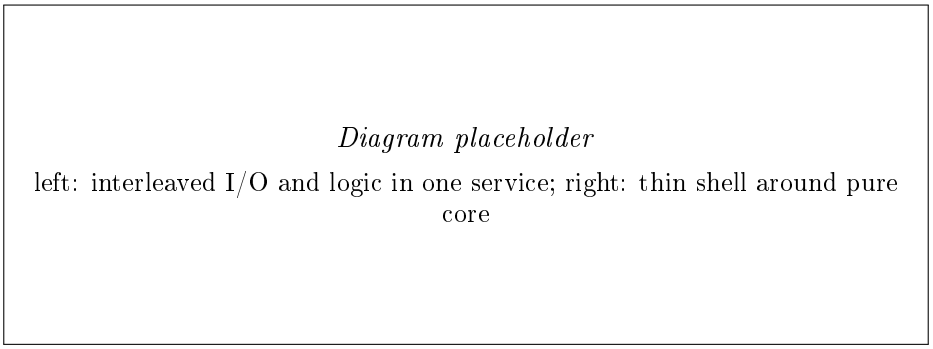

Figure 2.2: Fat service versus imperative sandwich structure (diagram to be added).

2.3.1 Sandwich in Pseudocode

The following pseudocode (adapted from common FCIS presentations) shows the rhythm for processing an order status. The Java examples later follow the same three steps.

```
// Imperative shell
void processOrder(OrderId orderId)
{
    Order order = db.find(orderId);

    OrderStatus newStatus = calculateNextStatus(
        order.status(),
        order.amount()
    );

    db.updateStatus(orderId, newStatus);
}

// Functional core
OrderStatus calculateNextStatus(
    OrderStatus status,
    double amount
)
{
    if (status == PENDING && amount > 100.0)
    {
        return REVIEW_REQUIRED;
    }

    return APPROVED;
}
```

Code snippet 2.3: Imperative sandwich in pseudocode

Do not pass heavy domain objects with hidden behaviour into the core if you can avoid it. Prefer simple records or maps that the core can treat as dumb data.[4] The shell is responsible for mapping from persistence entities and framework types into those records.

2.4 Example: Triangulate Kaiju Position (SCN-03)

This example revisits the triangulation use case from Chapter ??—our **known** example. We suspect a Kaiju has left a Breach; multiple sensors in the area return uncertain readings. Each reading is an *annular sector*—a direction and distance range, not a single point. We fuse several sectors to narrow the likely position to a small `EstimatedArea` polygon.
In the hexagonal chapter, ports and adapters framed the problem; fusion math remained a private stub inside `TriangulateKaijuPositionService`. FCIS reframes that stub as the functional core and leaves the service as a thin imperative shell.

2.4.1 Use Case Summary

The input is a suspected `Location` and a search `radius`. The output is an `EstimatedArea`: a list of `Location` points outlining a polygon. Sensors within the radius are loaded, polled through type-specific drivers, and converted to `AnnularSector` values. The geometric fusion is pure mathematics on those sectors; everything involving the ocean, hardware, or database is shell work.

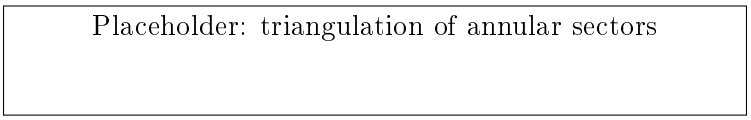

Figure 2.3: Triangulation of multiple annular sectors to estimate Kaiju position.

Figure 2.3 illustrates the idea: each sensor contributes an uncertain region; intersection narrows the Kaiju's likely position. For annular-sector terminology, see Chapter ?? and Figure ?? there.

Replace placeholder with a domain-specific triangulation diagram illustrating the intersection of annular sectors. Reference annular sector diagram from hexagonal chapter.

2.4.2 The Problem: Logic Inside the Service

Recall Code snippet ?? from Chapter ??: `computeEstimatedArea` is pure logic trapped inside an impure service class. It is hard to unit test without constructing the whole service and mocking three ports, even though the geometry only needs a `List<AnnularSector>`.
The FCIS refactor extracts that helper into a dedicated **functional core** class and leaves the service as a **thin imperative shell**.

2.4.3 Functional Core: KaijuPositionEstimator

We place pure types and logic under `com.abyssalwatch.core.triangulation`. `AnnularSector` is a record describing one sensor's uncertain region relative to a target. `KaijuPositionEstimator` exposes a single honest entry point.

```java
public record AnnularSector(
    Location sensorLocation,
    double bearingDegrees,
    double minDistanceMetres,
    double maxDistanceMetres,
    double spreadDegrees
)
{
}

public final class KaijuPositionEstimator
{
    private KaijuPositionEstimator()
    {
    }

    public static List<Location> estimateArea(
        List<AnnularSector> sectors
    )
    {
        if (sectors.isEmpty())
        {
            return List.of();
        }

        // Simplified stub: real fusion would intersect sectors.
        return sectors.stream()
            .map(KaijuPositionEstimator::centrePoint)
            .distinct()
            .toList();
    }

    private static Location centrePoint(AnnularSector sector)
    {
        double midDistance =
            (sector.minDistanceMetres()
                + sector.maxDistanceMetres()) / 2.0;

        return Geo.offset(
            sector.sensorLocation(),
            sector.bearingDegrees(),
            midDistance
        );
    }
}
```

Code snippet 2.4: `KaijuPositionEstimator` functional core (simplified fusion)

The implementation is deliberately a stub—the book is about architecture, not computational geometry—but the *boundary* is real: no database, no sensor driver, no logging. Production code would replace the body of `estimateArea` with a proper intersection algorithm while keeping the signature unchanged.

2.4.4 Imperative Shell: TriangulateKaijuPositionService

The shell class lives under `com.abyssalwatch.shell.triangulation`. It depends on outbound ports (or DAOs) for I/O and calls the core for fusion.

```java
public final class TriangulateKaijuPositionService
    implements TriangulateKaijuPositionPort
{
    private final SensorRepository sensorRepository;
    private final SensorDriverFactory sensorDriverFactory;

    public TriangulateKaijuPositionService(
        SensorRepository sensorRepository,
        SensorDriverFactory sensorDriverFactory
    )
    {
        this.sensorRepository = sensorRepository;
        this.sensorDriverFactory = sensorDriverFactory;
    }

    @Override
    public EstimatedArea triangulate(
        Location location,
        double radius
    )
    {
        List<Sensor> sensors =
            sensorRepository.findAllByCircle(
                location,
                radius
            );

        List<AnnularSector> sectors =
            pollSensors(sensors, location);

        List<Location> polygon =
            KaijuPositionEstimator.estimateArea(sectors);

        return new EstimatedArea(polygon);
    }

    private List<AnnularSector> pollSensors(
        List<Sensor> sensors,
        Location target
    )
    {
        List<AnnularSector> sectors = new ArrayList<>();

        for (Sensor sensor : sensors)
        {
            SensorDriver driver =
                sensorDriverFactory.create(sensor.type());

            sectors.add(driver.poll(target));
        }

        return List.copyOf(sectors);
    }
}
```

Code snippet 2.5: Thin shell: read sensors, call core, return result

Read the method top to bottom as an imperative sandwich: `findAllByCircle` and `pollSensors` are reads; `KaijuPositionEstimator.estimateArea` is the pure compute step; returning `EstimatedArea` is the write to the caller (persistence of the estimate, if needed, would be another shell concern in a follow-up use case).

2.4.5 Testing the Core

Tests target the core directly. No `SensorRepository` fake is required to prove that three sectors produce a non-empty polygon.

```
class KaijuPositionEstimatorTest
{
    @Test
    void estimateArea_withThreeSectors_returnsNonEmptyPolygon()
    {
        Location origin = new Location(
            new Latitude(0.0),
            new Longitude(0.0)
        );

        List<AnnularSector> sectors = List.of(
            sector(origin, 45.0, 100.0, 200.0),
            sector(origin, 90.0, 100.0, 200.0),
            sector(origin, 135.0, 100.0, 200.0)
        );

        List<Location> polygon =
            KaijuPositionEstimator.estimateArea(sectors);

        assertFalse(polygon.isEmpty());
    }

    private static AnnularSector sector(
        Location sensor,
        double bearing,
        double minDist,
        double maxDist
    )
    {
        return new AnnularSector(
            sensor,
            bearing,
            minDist,
            maxDist,
            5.0
        );
    }
}
```

Code snippet 2.6: Unit test for `KaijuPositionEstimator` without mocks

If fusion rules change—new weighting for thermal sensors, for example—you edit `KaijuPositionEstimator` and its tests. Sensor driver protocols can change independently in the shell.

2.5 Example: Reconcile Mission Telemetry (SCN-04)

This is our **new** example. Scout Vessel `S-402 "Mantis"` had intermittent uplink during the `Retrieval` phase of a mission. Some observations synced over the air; the rest arrive when the vessel docks at the Command Carrier. Before the mission can move toward `Completed` (invariant INV-05), the system must produce one consistent observation timeline.
That merge-and-deduplicate pipeline is ideal FCIS material: load both batches in the shell, reconcile in the core, persist the result in the shell.

2.5.1 Use Case Summary

When a retrieval order closes the tracking window, operators need a single authoritative observation set. The workflow is:

1. **Read:** load observations already synced for the mission, plus the dock-upload batch from the returning Scout Vessel,
2. **Compute:** merge, sort by timestamp, and deduplicate overlapping readings,
3. **Write:** persist the reconciled timeline so downstream mission-closure checks can proceed.

DIAGRAM: telemetry reconciliation: stored batch + dock upload → TelemetryReconciler → persist

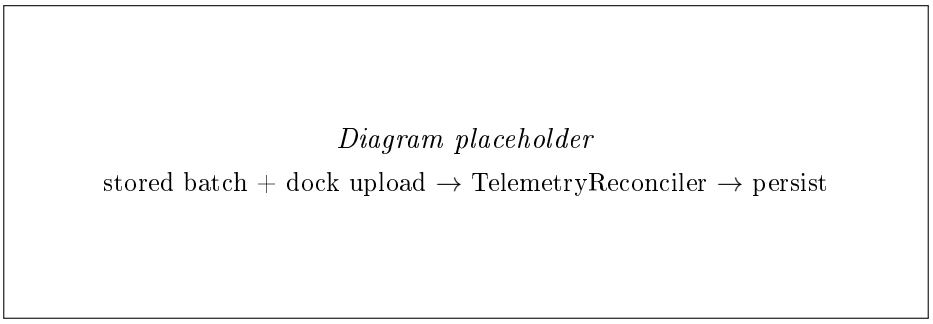

Figure 2.4: Telemetry reconciliation data flow (diagram to be added).

Figure 2.4 shows the imperative sandwich: two read paths feed one pure merge; the shell owns persistence and any mission-state transition.

2.5.2 Functional Core: TelemetryReconciler

Core types live under `com.abyssalwatch.core.telemetry`. `ObservationSnapshot` is the immutable input shape; the shell maps JDBC entities and dock-upload records to snapshots before calling the core.

```
public record ObservationSnapshot(
    ObservationId id,
    Instant observedAt,
    SensorType sensorType,
    double signalStrength
)
{
}

public record ReconciliationPolicy(
    Duration duplicateWindow
)
{
}

public record ReconciledTelemetry(
    List<ObservationSnapshot> observations,
    int duplicateCount,
    double completenessRatio
)
{
}

public final class TelemetryReconciler
{
    private TelemetryReconciler()
    {
    }

    public static ReconciledTelemetry merge(
        List<ObservationSnapshot> alreadySynced,
        List<ObservationSnapshot> dockUpload,
        ReconciliationPolicy policy
    )
    {
        List<ObservationSnapshot> combined =
            new ArrayList<>(alreadySynced);
        combined.addAll(dockUpload);

        combined.sort(Comparator.comparing(
            ObservationSnapshot::observedAt
        ));

        List<ObservationSnapshot> deduped =
            deduplicate(combined, policy.duplicateWindow());

        int duplicateCount =
            combined.size() - deduped.size();

        int expectedCount = combined.size();
        double completeness = expectedCount == 0
            ? 1.0
            : (double) deduped.size() / expectedCount;

        return new ReconciledTelemetry(
            List.copyOf(deduped),
            duplicateCount,
            completeness
        );
    }

    private static List<ObservationSnapshot> deduplicate(
        List<ObservationSnapshot> sorted,
```

Code snippet 2.7: `TelemetryReconciler` functional core

The deduplication rule keeps the stronger signal when two readings share a sensor type within the configured window. Production code might key on `ObservationId` or sensor serial numbers; here the focus is the architectural boundary: no DAO, no file reader, no mission-state update inside the core.

2.5.3 Imperative Shell: ReconcileMissionTelemetryService

The shell loads both batches, maps to snapshots, calls the reconciler, and persists the merged result.

```
public final class ReconcileMissionTelemetryService
{
    private final ObservationDao observationDao;
    private final DockUploadReader dockUploadReader;
    private final UnitOfWork unitOfWork;

    public ReconciledTelemetry reconcile(
        MissionId missionId,
        VesselId vesselId,
        ReconciliationPolicy policy
    )
    {
        List<ObservationSnapshot> synced =
            loadSynced(missionId);

        List<ObservationSnapshot> dockBatch =
            dockUploadReader.read(vesselId);

        ReconciledTelemetry merged =
            TelemetryReconciler.merge(
                synced,
                dockBatch,
                policy
            );

        persistMerged(missionId, merged);

        return merged;
    }

    private List<ObservationSnapshot> loadSynced(
        MissionId missionId
    )
    {
        return observationDao
            .findSyncedByMission(missionId)
            .stream()
            .map(this::toSnapshot)
            .toList();
    }

    private ObservationSnapshot toSnapshot(
        ObservationEntity row
    )
    {
        return new ObservationSnapshot(
            new ObservationId(row.getId()),
            row.getObservedAt(),
            new SensorType(row.getSensorType()),
            row.getSignalStrength()
        );
    }

    private void persistMerged(
        MissionId missionId,
        ReconciledTelemetry merged
    )
    {
        unitOfWork.execute(() ->
        {
            observationDao.replaceMissionObservations(
                missionId,
                merged.observations()
```

Code snippet 2.8: `ReconcileMissionTelemetryService` imperative shell

Whether the mission may transition toward `Completed` is a separate shell concern: it would read `ReconciledTelemetry.completenessRatio()` and vessel recovery state, then apply INV-05. The merge rules themselves stay in `TelemetryReconciler`.

2.5.4 Testing the Core with Table-Driven Cases

Table-driven tests document reconciliation behaviour without a database.

```
class TelemetryReconcilerTest
{
    private static final ReconciliationPolicy POLICY =
        new ReconciliationPolicy(Duration.ofMinutes(5));

    @Test
    void merge_duplicateWithinWindow_keepsStrongerSignal()
    {
        Instant t = Instant.parse("2026-06-01T12:00:00Z");

        List<ObservationSnapshot> synced = List.of(
            snap("obs-1", t, "Radiation", 30.0)
        );

        List<ObservationSnapshot> dock = List.of(
            snap("obs-2", t.plusSeconds(60), "Radiation", 45.0)
        );

        ReconciledTelemetry result =
            TelemetryReconciler.merge(synced, dock, POLICY);

        assertEquals(1, result.observations().size());
        assertEquals(45.0, result.observations().get(0)
            .signalStrength());
        assertEquals(1, result.duplicateCount());
    }

    @Test
    void merge_dockOnlyRecords_appended()
    {
        Instant t = Instant.parse("2026-06-01T12:00:00Z");

        List<ObservationSnapshot> synced = List.of(
            snap("obs-1", t, "Radiation", 30.0)
        );

        List<ObservationSnapshot> dock = List.of(
            snap("obs-2", t.plusHours(1), "Acoustic", 20.0)
        );

        ReconciledTelemetry result =
            TelemetryReconciler.merge(synced, dock, POLICY);

        assertEquals(2, result.observations().size());
    }

    @Test
    void merge_emptyDock_returnsSyncedUnchanged()
    {
        Instant t = Instant.parse("2026-06-01T12:00:00Z");

        List<ObservationSnapshot> synced = List.of(
            snap("obs-1", t, "Radiation", 30.0)
        );

        ReconciledTelemetry result =
            TelemetryReconciler.merge(
                synced,
                List.of(),
                POLICY
            );

        assertEquals(1, result.observations().size());
```

Code snippet 2.9: Table-driven tests for telemetry reconciliation

Changing the deduplication window or the keep-stronger rule is a one-file change with tests that run in milliseconds.

IN-REF: property-based testing chapter for fuzzing merged ObservationSnapshot lists

2.6 Recommended Package Layout

FCIS benefits from explicit package boundaries. The rule is simple: `core` must not depend on `shell`, `persistence`, or `presentation`. The shell depends on the core and on infrastructure.

DIAGRAM: FCIS package boundary: core has no outward infra imports; shell depends on core

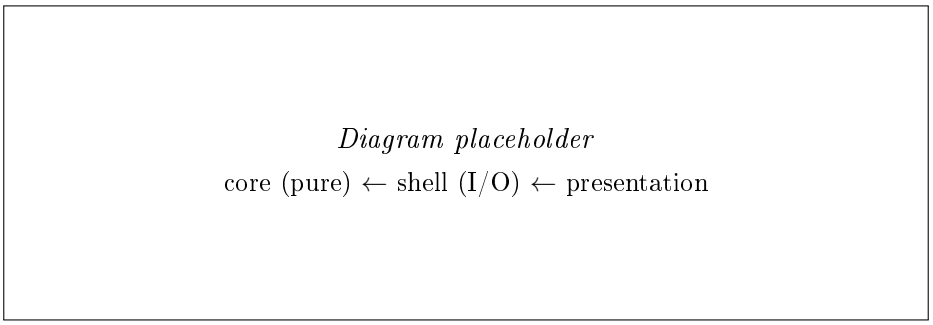

Figure 2.5: FCIS package dependency direction (diagram to be added).

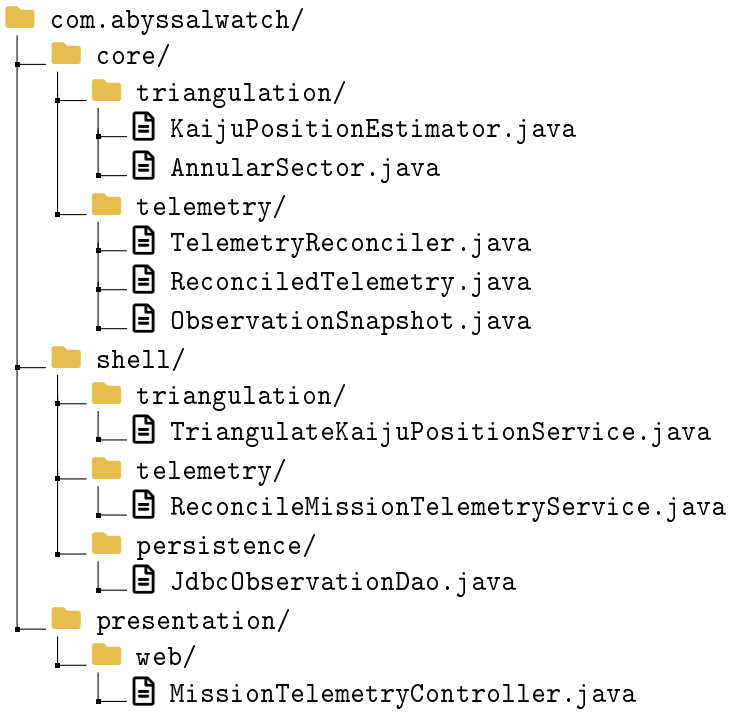

Figure 2.6: FCIS-oriented package layout for triangulation and telemetry examples.

Figure 2.6 groups code by FCIS role rather than only by technical layer. You can combine this with layered or hexagonal folders: for example, `shell.triangulation` may still contain `adapter` sub-packages behind ports from Chapter **??**. The invariant is that `core` never imports them.

Mapping entities at the boundary: persistence entities such as `ObservationEntity` stay in the shell. Map to `ObservationSnapshot` before calling `TelemetryReconciler`. Similarly,

`Sensor` domain records from the hexagonal example are shell-side; `AnnularSector` is the core input after polling.

2.7 Testing the Functional Core

2.7.1 What to Test Where

- **Functional core:** unit tests with fixed inputs; no Spring context, no test containers, no mocked repositories. Aim for branch coverage on rules and calculations.
- **Imperative shell:** integration tests that verify wiring—SQL mappings, transaction rollback, HTTP status codes. These tests are slower and fewer in number.
- **End-to-end:** optional smoke tests through the UI or API; not the primary place to prove reconciliation logic.

2.7.2 Contrast with Port-Based Testing

Hexagonal architecture (Chapter **??**) tests use cases by substituting fake outbound ports. That remains valuable for shell orchestration. FCIS reduces how much logic those fakes must exercise: if `KaijuPositionEstimator` is already proven, a triangulation integration test only needs to assert that the service calls the estimator and maps the result—not that intersection math is correct.

2.7.3 Guidelines

- Pass time, configuration thresholds, and feature flags *into* the core as parameters rather than reading static singletons.
- Prefer many small pure functions over one large `merge` method when branches multiply; each function gets its own focused test.
- When a shell test fails, determine whether the bug is in wiring (shell) or rules (core) before fixing; mixing fixes across the boundary recreates a fat service.
- Reconciliation cores such as `TelemetryReconciler` are good candidates for property-based tests later.

IN-REF: property-based testing chapter for merge invariants

2.8 Practical Trade-offs

Functional Core, Imperative Shell is a deliberate way to spend a little structure up front in exchange for testable calculations. It is not free, and it is not mandatory for every class.

2.8.1 Benefits

When the investment fits the problem, FCIS delivers:

- **Fast rule tests:** complex logic runs in unit tests without mocks, as in Code snippets 2.6 and 2.9,
- **Maintainability:** business rule changes concentrate in `core` packages, away from JDBC and HTTP,
- **Readability:** the imperative sandwich makes read-compute-write explicit, as in Figure 2.1,
- **Concurrency safety:** pure cores avoid shared mutable state during calculation,

- **Composable logic:** pure functions can be reused across use cases, batch jobs, and reporting pipelines without copying I/O code,
- and **Alignment with other styles:** FCIS sharpens hexagonal and layered boundaries rather than replacing them, as discussed in Section 2.2.4.

2.8.2 Costs

Those benefits come with costs teams should expect:

- **Extra types and mapping:** snapshots and DTOs at the core boundary add classes and conversion code,
- **Split methods:** one procedural service becomes a shell class plus one or more core classes,
- **Discipline required:** without import rules, `core` packages accumulate static database access and "temporary" logging,
- **Not every line benefits:** pass-through CRUD with no rules gains little from FCIS,
- and **Learning curve:** developers must recognise what belongs in the core versus the shell.

2.8.3 Things to Consider Before Deciding

Favour Functional Core, Imperative Shell when:

- calculations, scoring, reconciliation, or multi-step data transformations dominate a feature,
- rules change frequently and must be regression-tested,
- the same logic must run in the live app, batch jobs, and offline analysis,
- you need deterministic tests without standing up infrastructure,
- concurrent or parallel execution of business logic is on the roadmap,
- and existing services are hard to test because logic is interleaved with SQL and messaging.

Question or defer FCIS when:

- the feature is thin CRUD with at most one or two trivial branches,
- the team will not enforce package or import boundaries,
- you are building a throwaway prototype,
- all behaviour is already a single honest pure function with no I/O in the same class,
- and splitting would produce more boilerplate than testable logic.

When uncertain, extract one pure function from the hottest path—often a private helper in an oversized service—and measure whether tests become simpler. Deepen the split only where the payoff is clear.

2.8.4 Common Implementation Mistakes

The first group of mistakes weakens purity in the core:

- logging or metrics inside core methods,
- injecting repositories or HTTP clients into core classes,
- reading configuration singletons or environment variables in the core instead of passing values as parameters,
- mutating input lists or records passed from the shell,
- and calling `Instant.now()` inside the core rather than accepting time as an argument.

The next group leaves too much logic in the shell:

- long `if/else` chains in services that should be pure functions,
- scoring duplicated across REST controllers and batch importers instead of shared core code,
- returning different DTO shapes from the core (the core should return domain-neutral values; the shell maps to API contracts),
- and "anemic shell" services that only delegate while the core performs I/O through static helpers—a fake split.

Finally, some mistakes come from structure without discipline:

- creating `core` and `shell` packages but allowing cross-imports in both directions,
- passing live domain entities with behaviour into the core when a snapshot record would suffice,
- testing only through HTTP while complex math remains untested at the unit level,
- and applying FCIS to every class uniformly instead of focusing on calculation-heavy areas.

Most mistakes boil down to the same rule: the core must be honest and pure; the shell owns the messy world. When that holds, both Abyssal Watch examples in this chapter stay easy to evolve—deduplication rules in `TelemetryReconciler`, better fusion in `KaijuPositionEstimator`, without rewiring the ocean.

2.8.5 Closing Thoughts

We opened with functional programming vocabulary—pure and impure functions, honest signatures, immutable values, and side effects—then applied those ideas through the imperative sandwich. The triangulation example (Section 2.4) moved geometric fusion into `KaijuPositionEstimator`; the telemetry example (Section 2.5) moved merge and deduplication into `TelemetryReconciler`. In both cases the shell reads and writes; the core computes.
FCIS works in Java without adopting a new framework. Records, static pure methods, explicit packages, and disciplined imports are enough to start. Combine FCIS with the architectural styles elsewhere in this book: layers and hexagons tell you how to organise modules and ports; FCIS tells you how to keep the behaviour inside them testable and honest.

Bibliography

[1] Gary Bernhardt. *Boundaries*. Destroy All Software. 2012. URL: https://www.destroyallsoftware.com/talks/boundaries (visited on 06/09/2026).

[2] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional, 2002. ISBN: 978-0321127426.

[3] Martin Fowler. *Refactoring: Improving the Design of Existing Code*. 2nd ed. Addison-Wesley Professional, 2018. ISBN: 978-0134757599.

[4] Rico Fritzsche. *Applying Functional Core and Imperative Shell in Practice*. 2026. URL: https://ricofritzsche.me/applying-functional-core-and-imperative-shell-in-practice/ (visited on 06/09/2026).

[5] Google Testing Blog. *Simplify Your Code: Functional Core*. 2025. URL: https://testing.googleblog.com/2025/10/simplify-your-code-functional-core.html (visited on 06/09/2026).

[6] Microsoft Corporation. *Layered Application*. Patterns & Practices. 2003. URL: https://learn.microsoft.com/en-us/previous-versions/msp-n-p/ff650258(v=pandp.10) (visited on 06/04/2026).

[7] Microsoft Corporation. *Using a Three-Tier Architecture Model*. 2026. URL: https://learn.microsoft.com/en-us/windows/win32/cossdk/using-a-three-tier-architecture-model (visited on 06/04/2026).

[8] Mark Seemann. *Functional Core, Imperative Shell*. 2026. URL: https://functional-architecture.org/functional_core_imperative_shell/ (visited on 06/09/2026).

[9] SSENSE Tech. *A Look at the Functional Core and Imperative Shell Pattern*. 2026. URL: https://medium.com/ssense-tech/a-look-at-the-functional-core-and-imperative-shell-pattern-be2498da153a (visited on 06/09/2026).